

ODTE—An ensemble of multi-class SVM-based oblique decision trees

Ricardo Montaña^a, José A. Gámez^{a,*}, José M. Puerta^a

^aDepartamento de Sistemas Informáticos., Universidad de Castilla-La Mancha, Albacete, 02071, Spain

ARTICLE INFO

Keywords:

Oblique decision trees
Supervised classification
SVM
Ensemble
Multiclass strategies

ABSTRACT

We propose ODTE, a new ensemble that uses oblique decision trees as base classifiers. Additionally, we introduce STree, the base algorithm for growing oblique decision trees, which leverages support vector machines to define hyperplanes within the decision nodes. We embed a multiclass strategy (one-vs-one or one-vs-rest) at the decision nodes, allowing the model to directly handle non-binary classification tasks without the need to cluster instances into two groups, as is common in other approaches from the literature. In each decision node, only the best-performing model (SVM)—the one that minimizes an impurity measure for the n -ary classification—is retained, even if the learned SVM addresses a binary classification subtask. An extensive experimental study involving 49 datasets and various state-of-the-art algorithms for oblique decision tree ensembles has been conducted. Our results show that ODTE ranks consistently above its competitors, achieving significant performance gains when hyperparameters are carefully tuned. Moreover, the oblique decision trees learned through STree are more compact than those produced by other algorithms evaluated in our experiments.

1. Introduction

Classification is a problem which basically consists into assigning a label to a given instance or example from a predefined and finite set of labels. Formally, objects or instances are defined over a set of features or attributes $\mathcal{X} = \{X_1, \dots, X_n\}$, each one taking values in a predefined domain $\Omega(X_i)$ which can be of numerical or discrete nature. On the other hand, the target or *class* variable Y is of discrete nature and take values in a finite and mutually exclusive set of labels $\Omega(Y) = \{y_1, \dots, y_k\}$. Then, a *classifier* can be viewed as a function $f : \mathcal{X} \rightarrow Y$. In machine learning (ML) or data mining (DM) (Zaki & Meira, 2020) the goal is to learn this function or classifier from data.

Supervised learning is a main task in ML whose objective is learning a classifier from previously labeled data, i.e. objects or instances for which the correct class or label is known. This kind of data, $\mathbf{D} = \{(\bar{x}_i, y_i)\}_{i=1}^m$, usually comes from historical cases which have been previously solved, and the availability of the *correct* label y_i for each n -dimensional object $\bar{x}_i$ helps the learning algorithm to *supervise* the learning process, trying to minimize the error made on the given instances, whose correct label is known (Aggarwal, 2014).

A plethora of different paradigms to cope with the supervise classification problem is available in the literature: instance-based methods (Zhang, Li, Zong, Zhu, & Wang, 2018), logistic regression (Hilbe, 2009), Bayesian network classifiers (Bielza & Larrañaga, 2014; Flores, Gámez, & Martínez, 2012), decision trees (Kotsiantis, 2013), support

vector machines (Cervantes, Garcia-Lamont, Rodríguez-Mazahua, & Lopez, 2020; Vapnik, 1995), neural networks (Zhang, 2000), ensembles (Rokach, 2010; Zhou, 2012), etc. Currently, there is no doubt that from the point of view of accuracy, methods based on neural networks, especially deep learning approaches (Bengio, Lecun, & Hinton, 2021; Goodfellow, Bengio, & Courville, 2016), and ensembles (multiclassifiers), are the most competitive ones. Nonetheless, these two outstanding approaches, while being not mutually exclusive, have their own niche of application. Thus, deep learning-based methods excel in the processing of unstructured data (image, video, etc.) and require large amounts of data to work properly. On the other hand, ensembles tend to be used more with structured (tabular) data and, in general, can work with moderate or even small amounts of data (Shwartz-Ziv & Armon, 2022). In this paper we consider the case of structured data, so we focus on the use of ensembles as ML paradigm.

In essence, an ensemble is a multiclassifier, that is, a set of models is learned from different samples of the data, and new cases are classified by all of them and their results aggregated to provide a final output (Sagi & Rokach, 2018). The use of different strategies when learning the models and aggregating their results, give rise to different ensemble methods: bagging (Breiman, 1996), boosting (Schapire, 2003), stacking (Ting & Witten, 1999), etc. These strategies are usually generic, i.e. they can be instantiated with any base classifier, although there are also ensemble models based on a specific type of classifier, such as

* Corresponding author.

E-mail addresses: jose.gamez@uclm.es (J.A. Gámez), jose.puerta@uclm.es (J.M. Puerta).

Random Forest (Breiman, 2001) or Gradient Boosting (Chen & Guestrin, 2016; Friedman, 2001), which use a classification or regression trees as a base model (Breiman, Friedman, Olshen, & Stone, 1984; Quinlan, 1993). Even in the case of generic classifiers, decision trees are also often the most widely used choice as a base classifier, due to the fact that they are models with low bias and high variance, which is exactly what is usually reduced by means of the ensemble-based aggregation, thus resulting in a more robust and stable classifier. In this study we focus on the use of decision trees as the base models for the ensemble.

Decision Trees (DT) (Kotsiantis, 2013) are model-based classifiers whose inner structure is a tree. Basically, inner (or decision) nodes represent questions posed over the predictive attributes or features, while leaf nodes contain the class label to be returned as output. When a new instance $\vec{x}$ must be classified, the tree is traversed from the root to a leaf by choosing for each decision node the appropriate branch according to node question and the instance values for predictive attributes. Once a leaf node is reached, its associated class label is returned as outcome. The structural simplicity of DTs is one of its main advantages, as each path from the root to a leaf can be considered as a human-readable rule, which makes a DT an interpretable model able to justify its decisions (Lundberg et al., 2020). Furthermore, despite the fact of its simplicity, DTs performance is competitive with other state of the art classifiers, having been successfully and recently applied e.g. to real-time estimation of human pose by Microsoft Kinect (Shotton et al., 2013).

In literature we can find different families of decision trees, which mainly differ in the type of question/test included in the decision nodes, which in turn translate into different geometrical partitions of the representation (input) space. The most standard DT models are the so-called orthogonal ones, which consist of placing decisions of the type, $X_i \leq t$, where X_i is an attribute or feature and $t \in \Omega(X_i)$ being a threshold.¹ In a decision tree, impurity measures the heterogeneity of classes within a node. A node is pure if all its elements belong to the same class, whereas it is impure if it contains a mix of classes. The goal of a decision tree learning algorithm is to reduce impurity at each split to improve classification performance. Well known algorithms used to induce this type of DTs are C4.5 (Quinlan, 1993) and CART (Breiman et al., 1984), that use information criteria like *entropy* and *gini* respectively to measure the impurity of the resulting partition and select the best splitting test (X_i, t) at each decision node. However, although being successful ML algorithms (Wu et al., 2007), this type of DTs has the limitation of dealing only with decision boundaries that are parallel to the axes (features), and when this is not the case their size grow because several consecutive tests are included in order to approximate non axis-parallel boundaries (see Fig. 1).

Oblique decision trees (ODT) (Murthy, Kasif, & Salzberg, 1994) were introduced to capture decision boundaries not parallel to the axes. In this way it is possible to obtain more compact and generally more accurate trees. To represent these type of boundaries, the tests associated with the decision nodes are more complex, involving a function $g(\cdot)$ defined over a subset of the attributes, which on the other hand makes the resulting models lose part of their interpretability (see right part of Fig. 1). Despite this, there are a few efforts in the literature that address this issue (Jungermann, Kretinsky, & Weininger, 2023), and model-agnostic can also be used (Bonifazi et al., 2024). In the original proposals of ODTs the function $g(\cdot)$ was a linear combination of features, but later more complex ML models like support vector machines (SVM) or neural networks (NN) have also been used (Guo & Gelfand, 1992; Yıldız & Alpaydin, 2001; Zhang & Suganthan, 2015). Although this paper focuses on oblique decision trees, these models are part of a broader family of tree-based classifiers known as Multivariate Decision Trees (MVDTs). MVDTs allow combinations of features

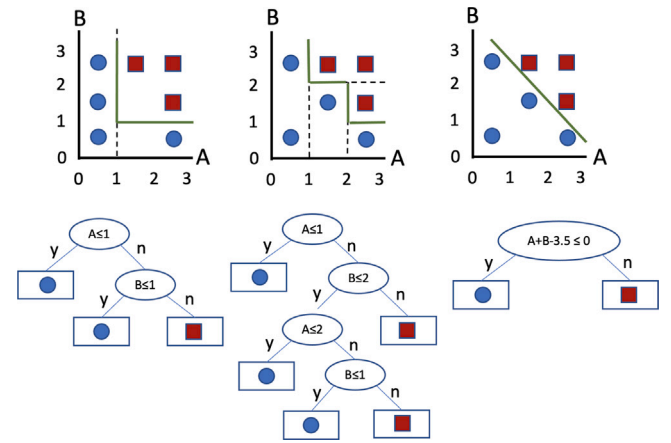


Fig. 1. Different decision boundaries and their corresponding decision trees: (left) axis-parallel decision boundary and simple DT; (middle) non axis-parallel decision boundary and complex DT because of the *ladder* effect; (right) non axis-parallel decision boundary and simple oblique DT.

to define splits at internal nodes. A detailed analysis of these algorithms can be found in Cañete-Sifuentes, Monroy, and Medina-Pérez (2021). One notable member of the MVDT family is Functional Trees, which enable combinations of features at both inner nodes and leaf nodes. In the algorithm proposed by Gama (2004), multivariate tests at decision nodes are created during tree growth, while functional leaves are incorporated during pruning.

Nowadays, ensemble algorithms constitute state of the art ML algorithms, usually being the first choice for practitioners to approach classification problems, especially in the case of structured (tabular) data.² Because of the success of ensemble techniques, research on ODTs and their use as base models in ensemble classifiers, has gained attention in recent years (Blanco, Japón, & Puerto, 2023; Ganaie, Tanveer, & Suganthan, 2020; Rodrigo, Alfaro, Aledo, & Gámez, 2024; Yang, Shen, & Gao, 2019). In this article we focus on the use SVM-based ODTs as base classifiers in ensemble algorithms. In order to do so, it is necessary to deal with three different problems or choices; (1) how to deal with domains where the class variable has more than two labels?; (2) what kind of SVM algorithm to use?; and (3) what ensemble technique(s) to use for classifier design?. On the contrary to recent proposals in this research line (see Section 2) that require internal transformations to actually work with binary class problems in the decision nodes, or the use of complex and sophisticated SVM models, our proposal focuses on their ease of use by practitioners, avoiding the internal transformation of the class variable and using standard SVM algorithms that can be found in commonly used ML libraries. In fact our proposed algorithm is available as an integrated classifier in ScikitLearn. The extensive experimental analysis carried out shows that the proposed ensemble, ODTE, outperforms alternative algorithms on average, standing out for its ease of use and its integration in standard software, which allows its parameters to be easily adjusted to adapt to different domains (datasets).

In summary, our main contributions in this study are:

- We introduce ODTE, an ensemble algorithm based on Oblique Decision Trees.
- We present STree, a novel algorithm for learning oblique decision trees that serve as the base classifiers in ODTE. This method effectively handles a multi-class target variable by producing a single model. The core idea behind the method is to embed a multiclass

¹ In this study we focus on the case of numerical attributes, that is, $\Omega(X_i) \subseteq \mathbb{R}$. Categorical variables could be easily accommodated by some kind of standard transformation.

² <https://www.kdnuggets.com/2015/12/harasyimiv-lessons-kaggle-machine-learning.html>.

strategy (One versus Rest (OvR) or One versus One (OvO)) within each internal node, selecting only one of the learned SVMs to guide the splitting process, specifically, the one that minimizes impurity for the n -ary classification task.

- The source code for both ODTE and STree, along with all datasets used in our study, is published on GitHub, PyPi and Zenodo to facilitate reproducibility in future research.

In the next sections we review recent research in the design of ODT-based ensembles (Section 2) and detail the main components of our proposal (Section 3). Then, in Section 4 we describe the experiments carried out to evaluate the proposed algorithm, which involve a significant benchmark of datasets and competing algorithms. Finally, our conclusions are presented in Section 5.

2. Related work

2.1. Oblique decision trees

Constructing a Decision Tree (DT) from data typically involves a recursive partitioning process that divides the data into multiple subsets based on a selected test (*split*) at each internal node. This recursive partitioning stops when the data arriving at a node predominantly belongs to a single category, at which point the node becomes a leaf. Thus, the critical aspect of the DT learning process lies in determining the appropriate test or division for an internal node. In the case of axis-parallel DTs, information theory-based or statistical metrics are employed to evaluate which test most significantly diminishes the uncertainty of the class variable. Commonly utilized measures include Shannon entropy (as seen in C4.5 Quinlan, 1993) and the Gini index (as used in CART Breiman et al., 1984).

In oblique Decision Trees (DTs), more sophisticated multivariate tests are utilized, resulting in models that are generally more compact and accurate. However, selecting the appropriate test for a given internal node is more computationally demanding. In the majority of Oblique DT (ODT) algorithms, the test comprises a linear combination of the input attributes, specifically, $\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n > 0$. The objective is to identify the β parameters that define the hyperplane, which, in turn, produces the binary partition that most effectively reduces the uncertainty of the class variable. In the CART-LC algorithm (Breiman et al., 1984), a coordinate descent method is employed for parameter optimization, whereas OC1 (Murthy et al., 1994) enhances this approach by incorporating multiple restarts and random perturbations to avoid local optima. Both CART-LC and OC1 start the optimization with the optimal axis-parallel partition. WODT (Yang et al., 2019) transforms the optimization challenge by adopting a continuous and differentiable weighted information entropy function as the objective, allowing for the use of gradient descent for optimization. Furthermore, WODT also distinguishes itself by initiating with a random hyperplane. Additionally, metaheuristic algorithms have been applied to overcome local optima in this context (Rivera-López, Canul-Reich, Gámez, & Puerta, 2017).

Recent advancements in ODTs have focused on enhancing optimality and scalability, as discussed in Hu, Rudin, and Seltzer (2019). Efforts have been made to develop sparse oblique trees that achieve low generalization error and offer rapid inference, as highlighted in Carreira-Perpinan and Tavallali (2018). Additionally, tackling imbalanced data has become a priority, with optimization strategies now being directed towards non-linear metrics to address this issue (Demirović & Stuckey, 2021).

2.2. SVM-based oblique decision trees

Beyond linear functions, advanced machine learning models such as neural networks or support vector machines (SVMs) have been explored as criteria for splits (Bennett & Blue, 1998; Kotschieder, Fiterau,

Criminisi, & Bulò, 2015), enabling the application of both multivariate linear and nonlinear tests. This paper concentrates on employing SVMs (Boser, Guyon, & Vapnik, 1992; Vapnik, 1995) to develop the tests for internal nodes. The conventional SVM algorithm aims to find the optimal separating hyperplane that maximizes the margin between the training data for a binary class variable. To address non-linearly separable challenges, the input vectors might be transformed into a high-dimensional feature space, thereby converting the problem into one of linear classification. This transformation and the solution process can be efficiently combined through the use of the kernel trick, which allows for direct computation with various types of kernels (e.g., linear, polynomial).

The literature showcases a variety of Oblique Decision Tree (ODT) methodologies that incorporate Support Vector Machines (SVMs) for determining the hyperplane at each internal node. Thus, traditional SVMs equipped with linear (Bennett & Blue, 1998), radial-basis function (Zhang, Zhou, Su, & Jiao, 2007), and polynomial kernels (Menkovski, Christou, & Efremidis, 2008) have been employed for this purpose. More sophisticated SVM algorithms, such as the multisurface proximal SVM (MPSVM) and the twin bounded SVM (TBSVM), are highlighted in Ganaie et al. (2020), Manwani and Sastry (2012) and Zhang and Suganthan (2015) respectively. Both MPSVM and TBSVM are designed to learn two hyperplanes, each one optimized to be closer to the data samples of one class and farther from those of the opposing class; classification of instances is then achieved by evaluating the distance of instances to both hyperplanes.

The algorithms in Blanco et al. (2023), Ganaie et al. (2020), Manwani and Sastry (2012), Menkovski et al. (2008) and Zhang and Suganthan (2015) demonstrate capability in addressing multi-class issues. In the approach discussed by Menkovski et al. (2008), a unique one-vs-rest binary problem is created for each class, leading to the development of k SVM models. This method involves constructing a vector of length k for every instance, where each dimension corresponds to the distance from the respective hyperplane. Subsequently, instances are categorized into r clusters via the X-means algorithm (Pelleg & Moore, 2000), with the cluster count r denoting the number of branches emanating from an internal node. In the methods proposed by Ganaie et al. (2020) and Zhang and Suganthan (2015), a method is used at internal nodes to split class labels into two groups based on Bhattacharyya distance, which then informs the binary classification tackled by MPSVM and TBSVM, respectively. Manwani and Sastry (2012) present a method where the multi-class issue is simplified to a binary challenge at each internal node by opposing the class with the largest number of instances against the aggregate of other classes. In Blanco et al. (2023) the authors work on multiclass categorization by introducing a mathematical programming model designed to optimize the internal structure of binary oblique decision trees. This model innovatively determines fictitious classes at the internal nodes, derived from the misclassification errors observed at the leaf nodes, thus ensuring the tree maintains a binary structure. Recently, Chen and Ge (2019) introduced a classification method utilizing SVM-trees combined with K-means to address the issue of imbalanced fault classification in industrial process applications, while Rodrigo et al. (2024) proposed a method to induce oblique label ranking trees to deal with the non-standard classification problem of label ranking.

2.3. Ensembles of oblique decision trees

Due to the rise of ensemble classifiers, oblique decision trees have naturally become prominent as base classifiers in these models. For instance, Tan and Dowe (2006) introduces a new decision forest learning scheme where the base learners are Minimum Message Length (MML) oblique decision trees. Zhan, Liu, and Xia (2024) study the consistency of combining oblique decision trees with boosting and random forests, and Cantu-Paz and Kamath (2003) applies evolutionary algorithms to construct ensembles of oblique decision trees.

Due to the affinity with our work, we highlight ensembles of oblique decision trees based on the use of SVMs. Notable examples include the models proposed in Ganaie et al. (2020), which are based on twin bounded SVMs, the models proposed in Menkovski et al. (2008), which incorporate heuristic techniques for enhancing the oblique decision tree construction process, and the models proposed in Zhang and Suganthan (2015), which use multi-surface proximal SVMs.

Finally, the importance and rise of ensemble models based on the use of oblique decision trees is demonstrated by their penetration in various application domains. Thus, ensembles of oblique decision trees have been considered for classifying gene expression data (Huynh, Nguyen, & Do, 2018), imbalanced fault classification in industrial processes (Chen & Ge, 2019), hyperspectral data classification (Poona, Van Niekerk, & Ismail, 2016), image classification (Singh Hada & Carreira-Perpiñán, 2022), generating explanations for neural network classifications (Hada, Carreira-Perpiñán, & Zharmagambetov, 2021), and developing resource-efficient models for neural signal classification (Zhu, Farivar, & Shoaran, 2020), among others.

3. Proposed method: ODTE

We introduce the Oblique Decision Tree Ensemble (ODTE), a bagging-inspired ensemble with oblique decision trees as base classifiers. These trees utilize an SVM to compute hyperplanes at each inner node. In general, a binary classification problem is addressed, requiring the learned hyperplane to separate the two classes. In our description, we will treat the learned classifier as a function rather than focusing on the hyperplane inherent to the classifier (SVM). While SVM primarily deals with binary classification, a key objective of our approach is to effectively handle multi-class target variables. Unlike traditional methods that implement one-vs-rest (OvR) or one-vs-one (OvO) strategies as a separate external process, our method integrates these strategies directly at each internal node of the tree. In our proposal, we select the classifier (hyperplane) that yields the best binary partition to be included as the conditional test in the node.

Through the use of the OvR or OvO strategies at each decision node, we eliminate the need for clustering the set of instances into two groups, as previously required in other studies (Ganaie et al., 2020; Menkovski et al., 2008; Zhang & Suganthan, 2015). Intuitively, grouping labels into a binary *artificial* class should result in more compact and balanced trees, however, our experiments show that segregating one label from the others as early as possible tends to produce more compact trees. Another critical aspect is our approach to select the hyperplane after facing one class label against the rest or one class label against other in a pairwise fashion. Unlike methods that typically favor the majority class label, we consider the distribution of the remaining labels across the two subsets of the partition. We then select the classifier (SVM) that results in the least impure partition, based on the distribution of the class labels in the obtained partition.

From a technical perspective, our ensemble algorithm (ODTE) (Algorithm 1), utilizes sampling with replacement to obtain the bootstrap datasets. Each (sampled) dataset is then used to train an SVM-based oblique decision tree by using our proposed STree algorithm (Algorithm 2). For inference, each ODT in the ensemble processes the given instance, and majority voting is employed to determine the final outcome (label).

In the rest of this section, we provide a detailed description of our proposed algorithm, STree,³ designed to learn an SVM-based ODT from data. As usual, STree is a recursive algorithm that begins by receiving a training dataset $\mathbf{D}' = \{(\vec{x}_i, y_i)\}_{i=1}^I$. Then, the method proceeds as follows:

1. If the stopping condition is met, e.g., max depth has been reached or there is almost only one class label in the sample,

a leaf node is created having as *outcome* the more frequent class label, i.e. the mode, in the dataset $\mathbf{D}'$.

2. Otherwise, a binary split is created using a classifier obtained by an SVM learning algorithm. Therefore, the inner classification problem(s) must be reformulated as a binary one. Basically, if the number of labels (k') in the sample is greater than 2 – notice that as the tree grows in depth, not all the labels will be present – in the received set $\mathbf{D}'$, then a multiclass classification strategy is employed. The classifier corresponding to the SVM whose application produces the partition with the higher information gain with respect to the class variable, is stored as the splitting test in the current node. The detailed steps of this process are outlined in algorithm 2, however, let us to highlight the main stages here:

- If $k' = 2$, we are dealing with a *binary classification* problem. The SVM algorithm is applied to learn the classifier (hyperplane), denoted as *model*. This classifier is then used to partition the instances in $\mathbf{D}'$ into two subsets: $\mathbf{D}'^+$ and $\mathbf{D}'^-$, based on whether the output of $model(\vec{x})$ assigns the label + or – to the instance $\vec{x}$. Specifically, the instances above the corresponding hyperplane are assigned the label +, while those below the hyperplane are assigned the label –. The classifier (*model*) is stored in the node to serve as the decision rule during inference (line 16, Algorithm 2).
- If $k' > 2$ we are in a *multi-class* setting. In this scenario, we resort to techniques that transform a multiclass classification problem into several binary classification problems, although in the end, we will select only one of these problems (models) to be included in our tree (line 18, Algorithm 2).

– *Learning stage.* Depending on the multiclass strategy used we have:

- * *One-vs-one* (OvO) strategy (Bishop, 2006, pg. 339) defines $r = \frac{k'(k'-1)}{2}$ binary classification problems on all possible pairs of classes: 1 (label 1 vs. label 2), 2 (label 1 vs. label 3), ..., $k' - 1$ (label 1 vs. label k'), ..., k' (label 2 vs. label 3), ..., r (label $k' - 1$ vs. label k'). Let $model_j$, $j = 1, \dots, r$, be the classifier learned by the SVM algorithm for the j th binary classification problem (label l vs. label s ; $l \neq s$), by using only the instances in $\mathbf{D}'$ labeled with $Y = l$ or $Y = s$ (line 20, Alg. 2).
- * *One-vs-rest* (OvR) strategy (Bishop, 2006, pg. 339), defines k' binary classification problems by considering, respectively, each label as the positive class and the union of the remaining labels as the negative class. Let $model_j$, $j = 1, \dots, k'$, be the classifier learned by the SVM algorithm for the j th binary classification problem (label j vs. all labels minus j) (line 22, Alg. 2).

– *Decision stage.* Let $\mathbf{D}'_j^+$ and $\mathbf{D}'_j^-$ denote the partitions generated by one of the learned models, $model_j$, from the SVM algorithm, when applied to all instances in $\mathbf{D}'$. In the case of the OvO approach, $\mathbf{D}'_j^+$ and $\mathbf{D}'_j^-$ correspond to the instances with predicted labels $Y = l$ or $Y = s$ (represented as + or –) for the learned model $model_j$. In the case of the OvR approach, $\mathbf{D}'_j^+$ and $\mathbf{D}'_j^-$ correspond to the instances with predicted labels j or $\neg j$ (represented as + or –) for the learned model $model_j$ (line 25, Alg. 2).

Let $impurity(Y, \mathbf{D})$ be a measure which evaluates the impurity of the class variable Y in $\mathbf{D}$. Then, we select

³ A preliminary version of STree was presented in Montañana, Gámez, and Puerta (2021).

the classifier $model_{b^*}$ such that

$$b^* = \arg \min_{j=1, \dots, \rho} \left(\frac{|D_j^+|}{|D|} \text{impurity}(Y, D_j^+) + \frac{|D_j^-|}{|D|} \text{impurity}(Y, D_j^-) \right),$$

with $\rho = k'$ or $\rho = r$ depending on the multiclass strategy, OvR or OvO, used in the learning stage. $model_{b^*}$ is then stored in the current node for future use in splitting and inference (line 25, Alg. 2).

Notice that in our pseudocode we actually maximize the gain with respect to the initial partition, which is equivalent to minimize the impurity measure.

- Once the classifier ($model$) and the corresponding partition (D^+, D^-) have been selected, two branches are created for this node: positive, for those instances being classified as + by $model$ and, negative, for those instances being classified as - by $model$. Two recursive calls are then launched with D^+ and D^- as the dataset of input instances respectively (lines 29–30, Alg. 2).

In our implementation we made the following design decisions:

- As there is no agreement about which kernel is better or worse for a specific domain, we allow the use of different kernels (linear, polynomial and Gaussian radial basis function).
- Shannon entropy is used as the impurity measure, and Information Gain is used to select the $model$ (SVM) that yields the purest partition.
- Although a pruning stage has not yet been designed for STree, a pre-pruning strategy can be implemented by setting a maximum depth threshold for the tree.

For inference, given an instance to be classified, the tree must be traversed from the root to a leaf node, whose associated label is returned as the outcome. At each inner/decision node, the stored classifier $model$ is used, and depending on the obtained label, + or -, the instance follows the *positive* or *negative* branch.

Algorithm 1 Oblique Decision Tree Ensemble (ODTE)

Input:

$D = \{(\vec{x}_i, y_i)\}_{i=1}^m$: Data
 n_{trees} : number of trees to learn

Output: $trees$: the ensemble

```

function build_ensemble( $D, n_{trees}$ )
   $trees \leftarrow []$ 
  for  $i \leftarrow 1$  to  $n_{trees}$  do
     $D_i \leftarrow \text{bootstrap\_samples}(D, m)$ 
     $trees[i] \leftarrow \text{STree}(D_i)$ 
  end for
  return  $trees$ 
end function

```

4. Experimental evaluation

In this Section we describe the comprehensive experiments carried out to evaluate our proposal.

4.1. Benchmark

For the evaluation of our algorithm in comparison with state-of-the-art methods, we use a benchmark of 49 datasets, previously used in Ganaie et al. (2020) to test their ensembles of oblique decision trees. Specifically, 45 of the selected datasets come from the UCI machine learning repository (Dua & Graff, 2017), while the other 4 (Id 27 to 30 in Table 1) are related to a fisheries fecundity problem (see Ganaie et al. (2020) for the details).

The Table 1 shows an identifier (Id) for each dataset, its name, the number of instances (m), features (n) and class labels (k).

Algorithm 2 STree (Multi-class SVM-based oblique decision tree)

Input:

1: $D' = \{(\vec{x}_i, y_i)\}_{i=1}^l$: Data

Output: $tree$: the root node of an SVM-based oblique decision tree

```

2: function STree( $D'$ )
  3: if stopping_condition( $\{y_i\}_{i=1}^l$ ) then
  4:   return create_leaf_node(mode( $\{y_i\}_{i=1}^l$ )) ▷ Leaf node
  5: end if
   $k' \leftarrow \text{num\_different\_labels}(\{y_i\}_{i=1}^l)$ 
   $r \leftarrow \frac{k'(k'-1)}{2}$ 
   $Y' \leftarrow \{y'_1, \dots, y'_k\}$  ▷ set of labels
   $I_{all} \leftarrow I(\{y_i\}_{i=1}^l)$  ▷  $I(\cdot)$  is an information theory measure
  10: if (OvO) then
  11:    $n_{models} \leftarrow r$ 
  12: else ▷ OvR
     $n_{models} \leftarrow k'$ 
  14: end if
  15: for  $j = 1$  to  $n_{models}$  do
  16:   if ( $j = k' = 2$ ) then break ▷ Two labels, one SVM is enough
  17:   end if
  18:   if (OvO) then
  19:     Let  $c_a$  and  $c_b$  the pair of labels corresponding to index  $j$ 
  20:      $models[j] \leftarrow \text{SVM}(D'^{c_a} \cup D'^{c_b})$ 
  21:   else ▷ OvR
     $models[j] \leftarrow \text{SVM}(D')$  using binary class  $y'_j$  (+) vs rest (-)
  23:   end if
   $D^+, D^- \leftarrow models[j](\vec{x}_i) \quad \forall (\vec{x}_i) \in D'$ 
   $I_j \leftarrow \frac{|D^+|}{|D|} I(\{y_i\}_{i=1}^l | D^+) + \frac{|D^-|}{|D|} I(\{y_i\}_{i=1}^l | D^-)$ 
   $IG_j \leftarrow I_{all} - I_j$ 
  27: end for
   $b^* \leftarrow \arg \max_{j=1, \dots, n_{models}} IG_j$ 
  29: if  $IG_{b^*} > 0$  then
  30:   node  $\leftarrow \text{create\_node}(models[b^*])$ 
  31:   node.left  $\leftarrow \text{STree}(D'^+)$ 
  32:   node.right  $\leftarrow \text{STree}(D'^-)$ 
  33: else
  34:   return create_leaf_node(mode( $\{y_i\}_{i=1}^l$ )) ▷ No split gain
  35: end if
  36: return node
37: end function

```

4.2. Algorithms

The following algorithms have been included in the comparison:

- ODTE. This is our proposed algorithm with default hyperparameters: 100 trees, STree as base estimator, do not limit number of features and do not limit the size of the bootstrap. Also default hyperparameters are used for STree.
- ODTE^T. This is the tuned version of our proposed algorithm. In this case the hyperparameters for the base estimators were optimized for each dataset using a gridsearch. The obtained values for each hyperparameter and dataset are available at <https://t.ly/rEzLH>.
- TBRaF (Ganaie et al., 2020). Twin Bounded Random Forest uses the fundamentals of Random Forests such as Bagging and random subspaces. Each tree is built with a bootstrap of the training data with a random subspace of features, and uses Twin Bonded Support Vector Machine (Shao, Zhang, Wang, & Deng, 2011) to generate the hyperplane needed, in this case 2 hyperplanes are generated, for splitting at each non-leaf node. The algorithm is run with a default parameter setting $c1 = 0.25$ and $c3 = 0.0625$.
- TBRaF^T. This is the tuned version of the previous model. The hyperparameters selected by the authors in the provided code are selected for each dataset (see <https://t.ly/zi-Ma>).

Table 1
Datasets used in the experiments.

Id	Dataset	<i>m</i>	<i>n</i>	<i>k</i>	Id	Dataset	<i>m</i>	<i>n</i>	<i>k</i>
1	balance-scale	625	4	3	26	musk-1	476	166	2
2	balloons	16	4	2	27	oocytes_merlucius_nucleus_4d	1022	41	2
3	breast-cancer	286	9	2	28	oocytes_merlucius_states_2f	1022	25	3
4	breast-cancer-wisc	699	9	2	29	oocytes_trisopterus_nucleus_2f	912	25	2
5	breast-cancer-wisc-diag	569	30	2	30	oocytes_trisopterus_states_5b	912	32	3
6	breast-cancer-wisc-prog	198	33	2	31	parkinsons	195	22	2
7	cardiotocography-10clases	2126	21	10	32	pima	768	8	2
8	cardiotocography-3clases	2126	21	3	33	pittsburg-bridges-MATERIAL	106	7	3
9	conn-bench-sonar-mines-rocks	208	60	2	34	pittsburg-bridges-REL-L	103	7	3
10	cylinder-bands	512	35	2	35	pittsburg-bridges-SPAN	92	7	3
11	dermatology	366	34	6	36	pittsburg-bridges-T-OR-D	102	7	2
12	echocardiogram	131	10	2	37	planning	182	12	2
13	fertility	100	9	2	38	post-operative	90	8	3
14	haberman-survival	306	3	2	39	seeds	210	7	3
15	heart-hungarian	294	12	2	40	statlog-australian-credit	690	14	2
16	hepatitis	155	19	2	41	statlog-german-credit	1000	24	2
17	ilpd-indian-liver	583	9	2	42	statlog-heart	270	13	2
18	ionosphere	351	33	2	43	statlog-image	2310	18	7
19	iris	150	4	3	44	statlog-vehicle	846	18	4
20	led-display	1000	7	10	45	synthetic-control	600	60	6
21	libras	360	90	15	46	tic-tac-toe	958	9	2
22	low-res-spect	531	100	9	47	vertebral-column-2clases	310	6	2
23	lymphography	148	18	4	48	wine	178	13	3
24	mammographic	961	5	2	49	zoo	101	16	7
25	molec-biol-promoter	106	57	2					

Table 2
Default STree hyperparameters.

Hyperparameter	Value
C	1
kernel	Linear
multiclass_strategy	OvO
splitter	Random
max_features	None
max_iter	1e5

- TBRoF (Ganaie et al., 2020). Twin Bounded Rotated Forest uses Principal Component Analysis (PCA) to rotate the dataset (Rodríguez, Kuncheva, & Alonso, 2006) and different rotation matrices are used to build each tree. The whole set of features are used to compute the hyperplanes in each node. The algorithm is run with a default parameter setting $c1 = 0.25$ and $c3 = 0.0625$.
- TBRoF^T. This is the tuned version of the previous model. The hyperparameters selected by the authors in the provided code are selected for each dataset (see <https://t.ly/WI8sS>).
- TBRRoF (Ganaie et al., 2020). Twin Bounded subrotation Forest Algorithm mixes PCA rotation and random subspaces, this way the rotation is applied only to the subset of features that is assigned for each tree. The algorithm is run with a default parameter setting $c1 = 0.25$ and $c3 = 0.0625$.
- TBRRoF^T. This is the tuned version of the previous model. The hyperparameters selected by the authors in the provided code are selected for each dataset (see <https://t.ly/gfr6T>).
- BaggingJ48-SVMODT. We have used Weka (Witten & Frank, 2005) Bagging Classifier to wrap J48-SVMODT (Menkovski et al., 2008) as base classifier. The Hedge algorithm proposed in Menkovski et al. (2008) that also uses J48-SVMODT as base classifier, has not been included in the experiments since it is clearly surpassed by BaggingJ48-SVMODT.
- PBC4cip. This is an ensemble based on multivariate decision trees that uses contrast patterns and a filtering algorithm (Loyola-

González et al., 2017). This algorithm⁴ has been compared against other multivariate decision trees in Cañete-Sifuentes, Monroy, Medina-Perez, Loyola-Gonzalez, and Vera Voronisky (2019), demonstrating very good performance, particularly with imbalanced datasets.

- BaggingWODT. We have used bagging classifier from scikit-learn (Pedregosa et al., 2011) to wrap WODT (Yang et al., 2019) as base classifier.
- RandomForest (Breiman, 2001) and XGBoost (Chen & Guestrin, 2016). State-of-the-art ensemble algorithms taken from scikit-learn.

As a common parameter, 100 trees are learned as base classifiers in all the ensemble algorithms, except for PBC4cip that uses 150 trees by default.

4.3. Reproducibility

ODTE and STree have been implemented in python as scikit-learn classifiers. For the sake of reproducibility and boosting future comparisons, we provide the code of both classifiers in: ODTE⁵ and STree.⁶ Furthermore, to make easier reproducing our experiments, the datasets have been uploaded to Zenodo.⁷

For the rest of algorithms we have used publicly available versions of RandomForest (python/scikit-learn) and XGBoost pre-built binary wheels for Python from PyPI (Python Package Index) have been used. The code for WODT (python), J48SVM-ODT (Java/Weka), PBC4cip (Java/Weka) and TBSVM-ODT (Matlab) have been provided to us by their authors. To create the ensembles for the base algorithms WODT and J48SVM-ODT we have wrapped them by using the public implementation of *bagging* available in scikit-learn and Weka respectively.

All the experiments have been run in an Intel(R) Xeon(R) CPU E5645 at 2.40 GHz running Linux operating system.

⁴ The implementation provided by PBC4cip algorithm has been used: <https://sites.google.com/view/leocanetesifuentes/home>.

⁵ <https://github.com/Doctorado-ML/ODte>.

⁶ <https://github.com/doctorado-ml/stree>.

⁷ <https://zenodo.org/records/13119174>.

Table 3

Accuracy results(mean $\pm$ std) for all the algorithms and datasets.

	Bagg-SVMODT	BaggingWodt	Odt	PBC4cip	RandomForest	TBRROF	TBRaF	TBRoF	XGBoost
1	0.9498 $\pm$ 0.018	0.9566 $\pm$ 0.018	0.9238 $\pm$ 0.022	0.9067 $\pm$ 0.025	0.8376 $\pm$ 0.020	0.9028 $\pm$ 0.021	0.9023 $\pm$ 0.024	0.6981 $\pm$ 0.231	0.8632 $\pm$ 0.026
2	0.5983 $\pm$ 0.219	0.6700 $\pm$ 0.247	0.6450 $\pm$ 0.304	0.6900 $\pm$ 0.271	0.6233 $\pm$ 0.285	0.6388 $\pm$ 0.236	0.5938 $\pm$ 0.230	0.5837 $\pm$ 0.241	0.6300 $\pm$ 0.305
3	0.7227 $\pm$ 0.051	0.7026 $\pm$ 0.053	0.7391 $\pm$ 0.026	0.6195 $\pm$ 0.072	0.7279 $\pm$ 0.046	0.7358 $\pm$ 0.040	0.7378 $\pm$ 0.046	0.6895 $\pm$ 0.050	0.6689 $\pm$ 0.054
4	0.9685 $\pm$ 0.012	0.9631 $\pm$ 0.012	0.9671 $\pm$ 0.012	0.9704 $\pm$ 0.013	0.9680 $\pm$ 0.012	0.9679 $\pm$ 0.014	0.9682 $\pm$ 0.012	0.9404 $\pm$ 0.016	0.9575 $\pm$ 0.014
5	0.9652 $\pm$ 0.018	0.9768 $\pm$ 0.012	0.9717 $\pm$ 0.015	0.9633 $\pm$ 0.014	0.9597 $\pm$ 0.018	0.9724 $\pm$ 0.014	0.9718 $\pm$ 0.013	0.9575 $\pm$ 0.016	0.9645 $\pm$ 0.019
6	0.8039 $\pm$ 0.061	0.7991 $\pm$ 0.051	0.7947 $\pm$ 0.049	0.6115 $\pm$ 0.066	0.7936 $\pm$ 0.030	0.8007 $\pm$ 0.058	0.7880 $\pm$ 0.061	0.7515 $\pm$ 0.060	0.7929 $\pm$ 0.045
7	0.8692 $\pm$ 0.014	0.8344 $\pm$ 0.020	0.8378 $\pm$ 0.017	0.8686 $\pm$ 0.016	0.8714 $\pm$ 0.018	0.6963 $\pm$ 0.025	0.6568 $\pm$ 0.058	0.7837 $\pm$ 0.018	0.8841 $\pm$ 0.015
8	0.9442 $\pm$ 0.011	0.9235 $\pm$ 0.013	0.9149 $\pm$ 0.013	0.9238 $\pm$ 0.012	0.9428 $\pm$ 0.011	0.8594 $\pm$ 0.019	0.8737 $\pm$ 0.018	0.8930 $\pm$ 0.014	0.9518 $\pm$ 0.008
9	0.7952 $\pm$ 0.071	0.8469 $\pm$ 0.057	0.7779 $\pm$ 0.060	0.8352 $\pm$ 0.056	0.8117 $\pm$ 0.053	0.8378 $\pm$ 0.047	0.8213 $\pm$ 0.051	0.7517 $\pm$ 0.053	0.8413 $\pm$ 0.055
10	0.7895 $\pm$ 0.036	0.7605 $\pm$ 0.045	0.7441 $\pm$ 0.043	0.7812 $\pm$ 0.043	0.7941 $\pm$ 0.039	0.7591 $\pm$ 0.039	0.7338 $\pm$ 0.045	0.6790 $\pm$ 0.043	0.7992 $\pm$ 0.031
11	0.9667 $\pm$ 0.025	0.9686 $\pm$ 0.018	0.9702 $\pm$ 0.016	0.9708 $\pm$ 0.017	0.9751 $\pm$ 0.014	0.9755 $\pm$ 0.015	0.9742 $\pm$ 0.015	0.9664 $\pm$ 0.018	0.9659 $\pm$ 0.019
12	0.8374 $\pm$ 0.065	0.7722 $\pm$ 0.074	0.8306 $\pm$ 0.052	0.7800 $\pm$ 0.070	0.8208 $\pm$ 0.062	0.8425 $\pm$ 0.056	0.8415 $\pm$ 0.055	0.7448 $\pm$ 0.070	0.8033 $\pm$ 0.074
13	0.8720 $\pm$ 0.069	0.8570 $\pm$ 0.056	0.8644 $\pm$ 0.035	0.6690 $\pm$ 0.089	0.8710 $\pm$ 0.038	0.8800 $\pm$ 0.060	0.8800 $\pm$ 0.059	0.7890 $\pm$ 0.088	0.8430 $\pm$ 0.052
14	0.7244 $\pm$ 0.053	0.6807 $\pm$ 0.052	0.7301 $\pm$ 0.016	0.6434 $\pm$ 0.050	0.6968 $\pm$ 0.049	0.7291 $\pm$ 0.042	0.7305 $\pm$ 0.043	0.7252 $\pm$ 0.046	0.6599 $\pm$ 0.048
15	0.8000 $\pm$ 0.052	0.8112 $\pm$ 0.049	0.8157 $\pm$ 0.045	0.8149 $\pm$ 0.051	0.8130 $\pm$ 0.039	0.8138 $\pm$ 0.038	0.8179 $\pm$ 0.041	0.7598 $\pm$ 0.045	0.7939 $\pm$ 0.048
16	0.7948 $\pm$ 0.071	0.8239 $\pm$ 0.086	0.8171 $\pm$ 0.065	0.7729 $\pm$ 0.075	0.8311 $\pm$ 0.054	0.8139 $\pm$ 0.050	0.8134 $\pm$ 0.057	0.7618 $\pm$ 0.062	0.7858 $\pm$ 0.058
17	0.6916 $\pm$ 0.037	0.7091 $\pm$ 0.031	0.7136 $\pm$ 0.008	0.6764 $\pm$ 0.041	0.7032 $\pm$ 0.038	0.7137 $\pm$ 0.034	0.7078 $\pm$ 0.031	0.6771 $\pm$ 0.038	0.6938 $\pm$ 0.039
18	0.9270 $\pm$ 0.029	0.9120 $\pm$ 0.034	0.9054 $\pm$ 0.030	0.9379 $\pm$ 0.026	0.9325 $\pm$ 0.024	0.9178 $\pm$ 0.025	0.9320 $\pm$ 0.024	0.8642 $\pm$ 0.033	0.9231 $\pm$ 0.028
19	0.9433 $\pm$ 0.037	0.9487 $\pm$ 0.031	0.9593 $\pm$ 0.041	0.9653 $\pm$ 0.033	0.9473 $\pm$ 0.040	0.9656 $\pm$ 0.027	0.9665 $\pm$ 0.025	0.9584 $\pm$ 0.027	0.9413 $\pm$ 0.037
20	0.7209 $\pm$ 0.028	0.7071 $\pm$ 0.030	0.7146 $\pm$ 0.027	0.7287 $\pm$ 0.024	0.7114 $\pm$ 0.026	0.3765 $\pm$ 0.090	0.7044 $\pm$ 0.030	0.6682 $\pm$ 0.102	0.7150 $\pm$ 0.026
21	0.7586 $\pm$ 0.051	0.8314 $\pm$ 0.041	0.8729 $\pm$ 0.041	0.8383 $\pm$ 0.045	0.8206 $\pm$ 0.039	0.8839 $\pm$ 0.035	0.8578 $\pm$ 0.040	0.7007 $\pm$ 0.052	0.7547 $\pm$ 0.055
22	0.8787 $\pm$ 0.033	0.8874 $\pm$ 0.030	0.9111 $\pm$ 0.023	0.8913 $\pm$ 0.027	0.8952 $\pm$ 0.025	0.8993 $\pm$ 0.025	0.8975 $\pm$ 0.024	0.8161 $\pm$ 0.030	0.8891 $\pm$ 0.022
23	0.8062 $\pm$ 0.076	0.8520 $\pm$ 0.058	0.8334 $\pm$ 0.066	0.8410 $\pm$ 0.054	0.8451 $\pm$ 0.053	0.8332 $\pm$ 0.059	0.8331 $\pm$ 0.063	0.7612 $\pm$ 0.071	0.8462 $\pm$ 0.058
24	0.8279 $\pm$ 0.023	0.7827 $\pm$ 0.023	0.8311 $\pm$ 0.026	0.8185 $\pm$ 0.021	0.7997 $\pm$ 0.030	0.8210 $\pm$ 0.022	0.8291 $\pm$ 0.022	0.8120 $\pm$ 0.022	0.7983 $\pm$ 0.027
25	0.8374 $\pm$ 0.079	0.8180 $\pm$ 0.083	0.7864 $\pm$ 0.081	0.8786 $\pm$ 0.076	0.8872 $\pm$ 0.067	0.8194 $\pm$ 0.075	0.8137 $\pm$ 0.073	0.6292 $\pm$ 0.083	0.8605 $\pm$ 0.073
26	0.8948 $\pm$ 0.029	0.8958 $\pm$ 0.036	0.8611 $\pm$ 0.039	0.8922 $\pm$ 0.032	0.8939 $\pm$ 0.032	0.8937 $\pm$ 0.026	0.8780 $\pm$ 0.032	0.8129 $\pm$ 0.036	0.8897 $\pm$ 0.031
27	0.8047 $\pm$ 0.024	0.8024 $\pm$ 0.021	0.8079 $\pm$ 0.022	0.8327 $\pm$ 0.023	0.8047 $\pm$ 0.021	0.8251 $\pm$ 0.023	0.8179 $\pm$ 0.022	0.8174 $\pm$ 0.022	0.8115 $\pm$ 0.025
28	0.9172 $\pm$ 0.021	0.9189 $\pm$ 0.017	0.9242 $\pm$ 0.013	0.9045 $\pm$ 0.016	0.9242 $\pm$ 0.018	0.9237 $\pm$ 0.015	0.9226 $\pm$ 0.015	0.9165 $\pm$ 0.017	0.9281 $\pm$ 0.018
29	0.8317 $\pm$ 0.027	0.8086 $\pm$ 0.024	0.8087 $\pm$ 0.024	0.8133 $\pm$ 0.029	0.8102 $\pm$ 0.023	0.8302 $\pm$ 0.024	0.8271 $\pm$ 0.025	0.7843 $\pm$ 0.030	0.8248 $\pm$ 0.028
30	0.9246 $\pm$ 0.018	0.9236 $\pm$ 0.017	0.9274 $\pm$ 0.017	0.9211 $\pm$ 0.018	0.9145 $\pm$ 0.020	0.9046 $\pm$ 0.019	0.9057 $\pm$ 0.019	0.9277 $\pm$ 0.015	0.9282 $\pm$ 0.017
31	0.9021 $\pm$ 0.040	0.9277 $\pm$ 0.035	0.9031 $\pm$ 0.050	0.8600 $\pm$ 0.065	0.9047 $\pm$ 0.051	0.9297 $\pm$ 0.036	0.9141 $\pm$ 0.036	0.8735 $\pm$ 0.045	0.9138 $\pm$ 0.043
32	0.7624 $\pm$ 0.032	0.7459 $\pm$ 0.031	0.7725 $\pm$ 0.025	0.7462 $\pm$ 0.032	0.7626 $\pm$ 0.023	0.7655 $\pm$ 0.028	0.7646 $\pm$ 0.029	0.7358 $\pm$ 0.030	0.7332 $\pm$ 0.030
33	0.8530 $\pm$ 0.067	0.8101 $\pm$ 0.074	0.8372 $\pm$ 0.050	0.7974 $\pm$ 0.073	0.8373 $\pm$ 0.052	0.8527 $\pm$ 0.053	0.8651 $\pm$ 0.058	0.7952 $\pm$ 0.079	0.8320 $\pm$ 0.058
34	0.6740 $\pm$ 0.101	0.6690 $\pm$ 0.088	0.6851 $\pm$ 0.081	0.6721 $\pm$ 0.115	0.6945 $\pm$ 0.091	0.6779 $\pm$ 0.084	0.6899 $\pm$ 0.078	0.5911 $\pm$ 0.097	0.6592 $\pm$ 0.101
35	0.6540 $\pm$ 0.113	0.6481 $\pm$ 0.112	0.6854 $\pm$ 0.089	0.6247 $\pm$ 0.105	0.6420 $\pm$ 0.080	0.6570 $\pm$ 0.089	0.6770 $\pm$ 0.079	0.5811 $\pm$ 0.091	0.6673 $\pm$ 0.091
36	0.8474 $\pm$ 0.076	0.8579 $\pm$ 0.078	0.8638 $\pm$ 0.051	0.7877 $\pm$ 0.087	0.8501 $\pm$ 0.045	0.8628 $\pm$ 0.056	0.8671 $\pm$ 0.064	0.8264 $\pm$ 0.075	0.8502 $\pm$ 0.050
37	0.7037 $\pm$ 0.056	0.6696 $\pm$ 0.072	0.7144 $\pm$ 0.010	0.4686 $\pm$ 0.070	0.6848 $\pm$ 0.043	0.7177 $\pm$ 0.057	0.7139 $\pm$ 0.060	0.6137 $\pm$ 0.074	0.6169 $\pm$ 0.070
38	0.6767 $\pm$ 0.081	0.5944 $\pm$ 0.087	0.7036 $\pm$ 0.054	0.3989 $\pm$ 0.100	0.6278 $\pm$ 0.094	0.6920 $\pm$ 0.085	0.7072 $\pm$ 0.084	0.5345 $\pm$ 0.101	0.5689 $\pm$ 0.085
39	0.9281 $\pm$ 0.044	0.9400 $\pm$ 0.041	0.9357 $\pm$ 0.028	0.9376 $\pm$ 0.031	0.9252 $\pm$ 0.034	0.9478 $\pm$ 0.029	0.9424 $\pm$ 0.030	0.9417 $\pm$ 0.031	0.9348 $\pm$ 0.038
40	0.6765 $\pm$ 0.035	0.6170 $\pm$ 0.034	0.6783 $\pm$ 0.003	0.5549 $\pm$ 0.036	0.6702 $\pm$ 0.024	0.6454 $\pm$ 0.030	0.6688 $\pm$ 0.030	0.5800 $\pm$ 0.040	0.6336 $\pm$ 0.032
41	0.7669 $\pm$ 0.022	0.7597 $\pm$ 0.027	0.7605 $\pm$ 0.018	0.7245 $\pm$ 0.027	0.7642 $\pm$ 0.021	0.7468 $\pm$ 0.026	0.7407 $\pm$ 0.030	0.7263 $\pm$ 0.025	0.7602 $\pm$ 0.023
42	0.8359 $\pm$ 0.043	0.8267 $\pm$ 0.045	0.8389 $\pm$ 0.043	0.8285 $\pm$ 0.042	0.8348 $\pm$ 0.050	0.8333 $\pm$ 0.042	0.8369 $\pm$ 0.039	0.7514 $\pm$ 0.054	0.8085 $\pm$ 0.047
43	0.9774 $\pm$ 0.006	0.9713 $\pm$ 0.007	0.9639 $\pm$ 0.008	0.9756 $\pm$ 0.007	0.9784 $\pm$ 0.008	0.9340 $\pm$ 0.024	0.9420 $\pm$ 0.021	0.9425 $\pm$ 0.070	0.9819 $\pm$ 0.007
44	0.7596 $\pm$ 0.028	0.7625 $\pm$ 0.028	0.8129 $\pm$ 0.030	0.7786 $\pm$ 0.026	0.7513 $\pm$ 0.026	0.7360 $\pm$ 0.038	0.7547 $\pm$ 0.030	0.7901 $\pm$ 0.027	0.7689 $\pm$ 0.027
45	0.9613 $\pm$ 0.022	0.9850 $\pm$ 0.010	0.9865 $\pm$ 0.009	0.9862 $\pm$ 0.010	0.9868 $\pm$ 0.009	0.9922 $\pm$ 0.007	0.9903 $\pm$ 0.008	0.9518 $\pm$ 0.019	0.9753 $\pm$ 0.012
46	0.9833 $\pm$ 0.008	0.9855 $\pm$ 0.009	0.9833 $\pm$ 0.009	0.9867 $\pm$ 0.010	0.9890 $\pm$ 0.009	0.8797 $\pm$ 0.029	0.8656 $\pm$ 0.025	0.9833 $\pm$ 0.008	0.9888 $\pm$ 0.008
47	0.8548 $\pm$ 0.043	0.8361 $\pm$ 0.043	0.8565 $\pm$ 0.039	0.8223 $\pm$ 0.038	0.8297 $\pm$ 0.036	0.8434 $\pm$ 0.038	0.8426 $\pm$ 0.037	0.8190 $\pm$ 0.041	0.8271 $\pm$ 0.036
48	0.7992 $\pm$ 0.020	0.9786 $\pm$ 0.022	0.9747 $\pm$ 0.021	0.9703 $\pm$ 0.033	0.9797 $\pm$ 0.024	0.9846 $\pm$ 0.017	0.9818 $\pm$ 0.020	0.9797 $\pm$ 0.026	0.9635 $\pm$ 0.031
49	0.9317 $\pm$ 0.047	0.9575 $\pm$ 0.038	0.9579 $\pm$ 0.048	0.9743 $\pm$ 0.035	0.9607 $\pm$ 0.040	0.9468 $\pm$ 0.045	0.9346 $\pm$ 0.056	0.9307 $\pm$ 0.052	0.9585 $\pm$ 0.035
Average	0.8346 $\pm$ 0.100	0.8303 $\pm$ 0.108	0.8391 $\pm$ 0.095	0.8093 $\pm$ 0.138	0.8347 $\pm$ 0.102	0.8261 $\pm$ 0.116	0.8306 $\pm$ 0.099	0.7871 $\pm$ 0.121	0.8256 $\pm$ 0.111

4.4. Results and analysis

To evaluate the performance of each pair (algorithm, dataset), we conducted a ten-times five-fold cross validation ($10 \times 5\text{cv}$). The same 10 seeds were used across all pairs to randomize the data prior to cross-validation. Since no severe imbalance is present in any dataset (see Ganaie et al. (2020, Table A1)), accuracy was used to compare the performance of the tested algorithms.

We distinguish two scenarios: all the algorithms are run by using a default setting for the hyperparameters (see Table 2); and some of the algorithms are run by using hyperparameter tuning. In this second case only our proposed algorithm (ODTE^T) and the algorithms for which the hyperparameter tuning is available (TBRaF^T, TBRoF^T and TBRROF^T) are considered. The mean and standard deviation over the 50 runs of the $10 \times 5\text{cv}$ are reported, respectively, in Tables 3 & 4.

As a first impression, looking at the tables we can observe that ODTE and ODTE^T are the winner algorithms both in number of wins and average accuracy. The difference is impressive in the scenario involving hyperparameter tuning, although all the algorithms are clearly benefited from the tuning process. To properly analyze the results,

we conducted a standard machine learning statistical analysis procedure (Demšar, 2006; García

Table 4

Accuracy results(mean $\pm$ std) for all the algorithms and datasets with optimized hyperparameters.

	ODTE ^T	TBRROF ^T	TBRaF ^T	TBRoF ^T
1	0.9750 $\pm$ 0.013	0.9044 $\pm$ 0.021	0.9023 $\pm$ 0.024	0.7787 $\pm$ 0.188
2	0.7250 $\pm$ 0.286	0.6288 $\pm$ 0.238	0.6000 $\pm$ 0.232	0.6188 $\pm$ 0.243
3	0.7517 $\pm$ 0.033	0.7347 $\pm$ 0.053	0.7331 $\pm$ 0.045	0.7227 $\pm$ 0.043
4	0.9707 $\pm$ 0.011	0.9688 $\pm$ 0.012	0.9690 $\pm$ 0.012	0.9618 $\pm$ 0.015
5	0.9761 $\pm$ 0.012	0.9741 $\pm$ 0.013	0.9714 $\pm$ 0.013	0.9734 $\pm$ 0.013
6	0.8031 $\pm$ 0.042	0.8115 $\pm$ 0.053	0.7999 $\pm$ 0.054	0.8043 $\pm$ 0.057
7	0.8405 $\pm$ 0.017	0.8327 $\pm$ 0.017	0.8324 $\pm$ 0.016	0.7827 $\pm$ 0.019
8	0.9264 $\pm$ 0.012	0.9222 $\pm$ 0.012	0.9197 $\pm$ 0.011	0.8954 $\pm$ 0.016
9	0.8587 $\pm$ 0.048	0.8357 $\pm$ 0.049	0.8216 $\pm$ 0.054	0.7905 $\pm$ 0.054
10	0.7842 $\pm$ 0.041	0.7728 $\pm$ 0.036	0.7510 $\pm$ 0.038	0.7148 $\pm$ 0.038
11	0.9806 $\pm$ 0.014	0.9752 $\pm$ 0.015	0.9741 $\pm$ 0.016	0.9732 $\pm$ 0.015
12	0.8550 $\pm$ 0.050	0.8507 $\pm$ 0.053	0.8496 $\pm$ 0.052	0.8292 $\pm$ 0.056
13	0.8806 $\pm$ 0.021	0.8810 $\pm$ 0.065	0.8804 $\pm$ 0.063	0.8800 $\pm$ 0.060
14	0.7350 $\pm$ 0.022	0.7374 $\pm$ 0.041	0.7356 $\pm$ 0.045	0.7326 $\pm$ 0.044
15	0.8232 $\pm$ 0.039	0.8183 $\pm$ 0.037	0.8203 $\pm$ 0.041	0.8068 $\pm$ 0.039
16	0.8425 $\pm$ 0.052	0.8180 $\pm$ 0.054	0.8195 $\pm$ 0.055	0.8099 $\pm$ 0.056
17	0.7316 $\pm$ 0.017	0.7133 $\pm$ 0.033	0.7143 $\pm$ 0.033	0.7072 $\pm$ 0.034
18	0.9498 $\pm$ 0.023	0.9471 $\pm$ 0.021	0.9321 $\pm$ 0.025	0.9069 $\pm$ 0.030
19	0.9653 $\pm$ 0.036	0.9653 $\pm$ 0.025	0.9630 $\pm$ 0.025	0.9772 $\pm$ 0.020
20	0.7275 $\pm$ 0.030	0.6862 $\pm$ 0.053	0.7126 $\pm$ 0.023	0.7029 $\pm$ 0.026
21	0.8727 $\pm$ 0.044	0.8851 $\pm$ 0.034	0.8527 $\pm$ 0.037	0.7556 $\pm$ 0.047
22	0.9115 $\pm$ 0.021	0.8994 $\pm$ 0.024	0.9007 $\pm$ 0.025	0.8686 $\pm$ 0.030
23	0.8636 $\pm$ 0.058	0.8559 $\pm$ 0.058	0.8485 $\pm$ 0.058	0.7734 $\pm$ 0.066
24	0.8353 $\pm$ 0.028	0.8261 $\pm$ 0.023	0.8296 $\pm$ 0.021	0.8130 $\pm$ 0.020
25	0.9014 $\pm$ 0.067	0.8146 $\pm$ 0.064	0.8230 $\pm$ 0.071	0.7480 $\pm$ 0.079
26	0.9332 $\pm$ 0.031	0.8939 $\pm$ 0.028	0.8807 $\pm$ 0.030	0.8629 $\pm$ 0.030
27	0.8411 $\pm$ 0.026	0.8389 $\pm$ 0.020	0.8391 $\pm$ 0.020	0.8256 $\pm$ 0.023
28	0.9326 $\pm$ 0.015	0.9270 $\pm$ 0.015	0.9238 $\pm$ 0.016	0.9195 $\pm$ 0.016
29	0.8557 $\pm$ 0.021	0.8365 $\pm$ 0.024	0.8345 $\pm$ 0.023	0.8049 $\pm$ 0.025
30	0.9309 $\pm$ 0.019	0.9323 $\pm$ 0.016	0.9279 $\pm$ 0.017	0.9304 $\pm$ 0.016
31	0.9344 $\pm$ 0.032	0.9310 $\pm$ 0.036	0.9143 $\pm$ 0.041	0.8948 $\pm$ 0.046
32	0.7727 $\pm$ 0.024	0.7685 $\pm$ 0.030	0.7633 $\pm$ 0.028	0.7675 $\pm$ 0.028
33	0.8732 $\pm$ 0.052	0.8602 $\pm$ 0.054	0.8667 $\pm$ 0.057	0.8014 $\pm$ 0.077
34	0.7061 $\pm$ 0.090	0.6904 $\pm$ 0.076	0.6956 $\pm$ 0.083	0.6185 $\pm$ 0.094
35	0.6942 $\pm$ 0.083	0.6924 $\pm$ 0.085	0.6865 $\pm$ 0.085	0.6222 $\pm$ 0.095
36	0.8686 $\pm$ 0.042	0.8702 $\pm$ 0.056	0.8650 $\pm$ 0.058	0.8714 $\pm$ 0.063
37	0.7309 $\pm$ 0.018	0.7202 $\pm$ 0.058	0.7146 $\pm$ 0.054	0.7131 $\pm$ 0.059
38	0.7121 $\pm$ 0.028	0.7109 $\pm$ 0.084	0.7074 $\pm$ 0.087	0.7115 $\pm$ 0.085
39	0.9605 $\pm$ 0.025	0.9468 $\pm$ 0.027	0.9439 $\pm$ 0.030	0.9507 $\pm$ 0.042
40	0.6783 $\pm$ 0.003	0.6783 $\pm$ 0.031	0.6783 $\pm$ 0.029	0.6783 $\pm$ 0.031
41	0.7682 $\pm$ 0.020	0.7578 $\pm$ 0.027	0.7548 $\pm$ 0.025	0.7561 $\pm$ 0.025
42	0.8481 $\pm$ 0.041	0.8367 $\pm$ 0.039	0.8374 $\pm$ 0.040	0.8250 $\pm$ 0.043
43	0.9711 $\pm$ 0.007	0.9760 $\pm$ 0.007	0.9698 $\pm$ 0.008	0.9568 $\pm$ 0.033
44	0.8133 $\pm$ 0.026	0.7863 $\pm$ 0.025	0.7765 $\pm$ 0.027	0.8033 $\pm$ 0.044
45	0.9910 $\pm$ 0.008	0.9920 $\pm$ 0.007	0.9913 $\pm$ 0.007	0.9807 $\pm$ 0.014
46	0.9939 $\pm$ 0.008	0.9857 $\pm$ 0.007	0.9799 $\pm$ 0.012	0.9833 $\pm$ 0.008
47	0.8635 $\pm$ 0.040	0.8505 $\pm$ 0.036	0.8501 $\pm$ 0.034	0.8505 $\pm$ 0.039
48	0.9844 $\pm$ 0.019	0.9826 $\pm$ 0.020	0.9817 $\pm$ 0.019	0.9879 $\pm$ 0.017
49	0.9707 $\pm$ 0.041	0.9466 $\pm$ 0.045	0.9411 $\pm$ 0.051	0.9462 $\pm$ 0.047
Average	0.8595 $\pm$ 0.093	0.8463 $\pm$ 0.097	0.8425 $\pm$ 0.097	0.8243 $\pm$ 0.103

Table 5

Results of the post-hoc test for the mean accuracy of the algorithms.

Classifier	pvalue	rank	win	tie	loss
Odte	–	3,88	–	–	–
TBRROF	1,0000e+00	3,95	27	0	22
TBRaF	1,0000e+00	4,19	27	0	22
RandomForest	8,5429e–01	4,47	27	0	22
BaggingJ48-SVMODT	4,8534e–01	4,73	31	0	18
BaggingWodt	8,2523e–02	5,20	32	0	17
XGBoost	3,8055e–02	5,39	32	0	17
PBC4cip	1,7426e–02	5,55	30	0	19
TBRoF	9,1628e–11	7,63	45	0	4

boldfaced. As can be observed, there is no statistically significant difference between ODTE and most of the algorithms, despite ODTE having a larger number of wins. The only algorithms that are clearly outperformed are XGBoost and TBRoF.

- In the hyperparameter tuning setting, Friedman test reports a p -value of 7,4815371e–16, thus rejecting the null hypothesis that

Table 6

Results of the post-hoc test for the mean accuracy of the algorithms with optimized hyperparameters.

Classifier	pvalue	rank	win	tie	loss
ODTE ^T	–	1,35	–	–	–
TBRROF ^T	7,6660e–04	2,22	40	0	9
TBRaF ^T	2,7947e–10	3,02	46	0	3
TBRoF ^T	7,9936e–15	3,41	44	0	5

Table 7

Averaged normalized tree size for each dataset and algorithm.

	ODTE	TBRROF	TBRoF	TBRaF	BaggJ48SVMODT	BaggWodt	PBC4cip
Size	1,00	3,87	2,77	2,82	2,66	11,73	–
Time	1,00	287,51	2,02	230,84	10,11	1,25	2,03

all the algorithms are equivalent. The results of the post hoc Holm's tests are shown in Table 6 by using ODTE^T as control, as it is the top ranked algorithms. As can be observed, in this case there is statistically significant difference between ODTE^T and the rest of studied algorithms.

To complete our analysis we also studied the complexity (size) of the obtained trees and also the training time required by the different oblique decision tree algorithms. For the sake of brevity⁸ in Table 7 we only show the values on average over the 49 datasets once we normalized them by using ODTE as control.

With respect to *size* we compute the average size (number of nodes) in the trees included in the ensemble,⁹ then we normalize by using ODTE. As all the numbers in the row are greater than one, it is clear that ODTE obtains the more compact trees. On the other hand, with respect to *CPU time* we must be careful with the analysis because the tested algorithms are in different programming languages. However, from the data in Table 7 we could obtain a similar conclusion, with ODTE being the faster algorithm.

In summary, considering all the evaluated dimensions (accuracy, size, and time), there is no doubt that ODTE emerges as the outstanding algorithm in the comparison, a fact that is even more pronounced when we take the hyperparameter tuning phase into account.

5. Conclusion

A new ensemble algorithm, ODTE, which incorporates a novel oblique decision tree model (STree) capable of directly handling a multi-class target variable, has been introduced. The STree algorithm generates a binary oblique decision tree that learns several SVM at each split, though only one is retained for inference. The experiments demonstrate that this approach performs well across a broad range of domains (49 datasets), surpassing competing ensembles of oblique decision trees and other state-of-the-art algorithms. It has also been observed that tuning the hyperparameters of the base algorithm and the ensemble for each dataset is crucial for achieving even better results, leading to a version of ODTE that significantly outperforms all other tested methods.

A limitation of our approach falls in the dimension of interpretability. Clearly, including an SVM in the decision nodes undeniably diminishes the interpretability of the resultant tree. Consequently, we believe that the dimension of explainability within the proposed STree model, akin to others emphasizing accuracy over transparency, necessitates attention through local and post-hoc explanation techniques. Obviously, this limitation of our base model (STree) extends to the proposed

⁸ Detailed values fore each algorithm and dataset can be observed in supplementary material: <https://t.ly/k1MGF> (size) and <https://t.ly/WCKxm> (time).

⁹ This data could not be included for PBC4cip because the package used did not provide the average size of the trees in the ensemble.

ensemble, ODTE, bringing it closer to the black-box family of classifiers. Fortunately, research in explainable AI (XAI) offers various techniques to shed light on the decisions made for specific instances (Guidotti et al., 2024; Lundberg & Lee, 2017; Ribeiro, Singh, & Guestrin, 2016).

For future research, we plan to go deeper in the use of (random/informed) sub-spaces, selecting features randomly as in random forest (Breiman, 2001) or using univariate or multivariate filter feature selection algorithms. Additionally, the benefits of fine-tuning hyperparameters have proven essential for the performance of the proposed algorithms, albeit at the cost of high computational CPU time. In future studies, we aim to investigate a type of lightweight auto-tuning integrated into the ODTE/STree algorithm(s).

CRedit authorship contribution statement

Ricardo Montaña: Data curation, Software, Visualization, Validation, Conceptualization, Methodology, Writing – original draft, Writing – review & editing. **José A. Gámez:** Supervision, Funding acquisition, Conceptualization, Validation, Writing – original draft, Writing – review & editing. **José M. Puerta:** Supervision, Funding acquisition, Conceptualization, Validation, Writing – original draft, Writing – review & editing.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used CHATGPT and GRAMMARLY in order to improve language and readability. After using these tools/services, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Jose A. Gamez, Jose M. Puerta reports financial support was provided by Spain Ministry of Science and Innovation. Jose A. Gamez, Jose M. Puerta reports financial support was provided by Junta de Comunidades de Castilla-La Mancha. Jose A. Gamez, Jose M. Puerta reports financial support was provided by European Regional Development Fund. Jose A. Gamez, Jose M. Puerta reports financial support was provided by University of Castilla-La Mancha. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

First of all, we are indebted to the authors of Ganaie et al. (2020), Menkovski et al. (2008) and Yang et al. (2019) for providing us with the code for their implementations. We are also grateful to the reviewers for their valuable comments. This work has been partially funded by the Government of Castilla-La Mancha and “ERDF A way of making Europe” under project SBPLY/21/180225/000062; by MCIU/AEI/10.13039/501100011033 and FEDER through project PID2022-139293NB-C32; and by the Universidad de Castilla-La Mancha and “ERDF A Way of Making Europe” under project 2022-GRIN-34437.

Data availability

The url links to download data and code have been included in the manuscript.

References

- Aggarwal, C. C. (2014). *Data classification: algorithms and applications* (1st ed.). Chapman & Hall/CRC.
- Arias, J., & Cózar, J. (2016). ExReport: Fast, reliable and elegant reproducible research. <https://cran.r-project.org/web/packages/exreport/index.html>. (Last Accessed 7 February 2025).
- Bengio, Y., Lecun, Y., & Hinton, G. (2021). Deep learning for AI. *Communications of the ACM*, 64(7), 58–65.
- Bennett, K. P., & Blue, J. A. (1998). A support vector machine approach to decision trees. vol. 3, In *1998 IEEE International Joint Conference on Neural Networks Proceedings* (pp. 2396–2401).
- Bielza, C., & Larrañaga, P. (2014). Discrete bayesian network classifiers: A survey. *ACM Computing Surveys*, 47(1).
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Blanco, V., Japón, A., & Puerto, J. (2023). Multiclass optimal classification trees with svm-splits. *Machine Learning*, 1–24.
- Bonifazi, G., Cauteruccio, F., Corradini, E., Marchetti, M., Terracina, G., Ursino, D., et al. (2024). A model-agnostic, network theory-based framework for supporting XAI on classifiers. *Expert Systems with Applications*, 241, Article 122588.
- Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on computational learning theory* (pp. 144–152).
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Monterey, CA: Wadsworth and Brooks.
- Cañete-Sifuentes, L., Monroy, R., & Medina-Pérez, M. A. (2021). A review and experimental comparison of multivariate decision trees. *IEEE Access*, 9, 110451–110479.
- Cañete-Sifuentes, L., Monroy, R., Medina-Perez, M. A., Loyola-Gonzalez, O., & Vera Voronisky, F. (2019). Classification based on multivariate contrast patterns. *IEEE Access*, 7, 55744–55762.
- Cantu-Paz, E., & Kamath, C. (2003). Inducing oblique decision trees with evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 7(1), 54–68.
- Carreira-Perpinan, M. A., & Tavallali, P. (2018). Alternating optimization of decision trees, with application to learning sparse oblique trees. vol. 31, In *Advances in neural information processing systems*.
- Cervantes, J., Garcia-Lamont, F., Rodríguez-Mazahua, L., & Lopez, A. (2020). A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*, 408, 189–215.
- Chen, G., & Ge, Z. (2019). SVM-tree and SVM-forest algorithms for imbalanced fault classification in industrial processes. *IFAC Journal of Systems and Control*, 8, Article 100052.
- Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794).
- Demirović, E., & Stuckey, P. J. (2021). Optimal decision trees for nonlinear metrics. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(5), 3733–3741.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.
- Dua, D., & Graff, C. (2017). UCI machine learning repository.
- Flores, M., Gámez, J., & Martínez, A. (2012). Supervised classification with Bayesian networks: A review on models and applications. (pp. 72–102).
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
- Gama, J. (2004). Functional trees. *Machine Learning*, 55(3), 219–250.
- Ganaie, M., Tanveer, M., & Suganthan, P. (2020). Oblique decision tree ensemble via twin bounded SVM. *Expert Systems with Applications*, 143, Article 113072.
- García, S., & Herrera, F. (2008). An extension on statistical comparisons of classifiers over multiple data sets for all pairwise comparisons. *Journal of Machine Learning Research*, 9, 2677–2694.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. The MIT Press.
- Guidotti, R., Monreale, A., Ruggieri, S., Naretto, F., Turini, F., Pedreschi, D., et al. (2024). Stable and actionable explanations of black-box models through factual and counterfactual rules. *Data Mining and Knowledge Discovery*, 38(5), 2825–2862.
- Guo, H., & Gelfand, S. (1992). Classification trees with neural network feature extraction. *IEEE Transactions on Neural Networks*, 3(6), 923–933.
- Hada, S. S., Carreira-Perpin'an, M. A., & Zharmagambetov, A. (2021). Sparse oblique decision trees: a tool to understand and manipulate neural net features. *Data Mining and Knowledge Discovery*, 1–40.
- Hilbe, J. M. (2009). *Logistic regression models*. Chapman & Hall/CRC.
- Hu, X., Rudin, C., & Seltzer, M. (2019). Optimal sparse decision trees. vol. 32, In *Advances in neural information processing systems*.
- Huynh, P.-H., Nguyen, V. H., & Do, T.-N. (2018). Random ensemble oblique decision stumps for classifying gene expression data. In *Proceedings of the 9th international symposium on information and communication technology* (pp. 137–144). New York, NY, USA: Association for Computing Machinery.

- Jungermann, F., Kretinsky, J., & Weininger, M. (2023). Algebraically explainable controllers: Decision trees and support vector machines join forces. *International Journal on Software Tools for Technology Transfer*, 25, 249–266.
- Kontschieder, P., Fiterau, M., Criminisi, A., & Bulò, S. R. (2015). Deep neural decision forests. In *2015 IEEE international conference on computer vision* (pp. 1467–1475).
- Kotsiantis, S. B. (2013). Decision trees: A recent overview. *Artificial Intelligence Review*, 39(4), 261–283.
- Loyola-González, O., Medina-Pérez, M. A., Martínez-Trinidad, J. F., Carrasco-Ochoa, J. A., Monroy, R., & García-Borroto, M. (2017). Pbc4cip: A new contrast pattern-based classifier for class imbalance problems. *Knowledge Based Systems*, 115, 100–109.
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2, 56–67.
- Lundberg, S. M., & Lee, S. (2017). A unified approach to interpreting model predictions. In *Advances in neural information processing systems 30: annual conference on neural information processing systems 2017, December 2017 4-9* (pp. 4765–4774). Long Beach, CA, USA.
- Manwani, N., & Sastry, P. S. (2012). Geometric decision tree. *IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics)*, 42(1), 181–192.
- Menkovski, V., Christou, I. T., & Efremidis, S. (2008). Oblique decision trees using embedded support vector machines in classifier ensembles. In *2008 7th IEEE international conference on cybernetic intelligent systems* (pp. 1–6).
- Montañana, R., Gámez, J. A., & Puerta, J. M. (2021). STree: a single multi-class oblique decision tree based on support vector machines. In *Advances in artificial intelligence - 19th conference of the spanish association for artificial intelligence, CAEPIA 2020/2021, Málaga, Spain, September 2021 22-24, proceedings* (pp. 54–64).
- Murthy, S. K., Kasif, S., & Salzberg, S. (1994). A system for induction of oblique decision trees. *Journal of Artificial Intelligence Research*, 2(1), 1–32.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Pelleg, D., & Moore, A. W. (2000). X-means: Extending k-means with efficient estimation of the number of clusters. In *Proceedings of the seventeenth international conference on machine learning* (pp. 727–734).
- Poona, N., Van Niekerk, A., & Ismail, R. (2016). Investigating the utility of oblique tree-based ensembles for the classification of hyperspectral data. *Sensors*, 16(11).
- Quinlan, J. R. (1993). *C4.5: programs for machine learning*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). Why should I trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, san francisco, CA, USA, August 2016 13-17* (pp. 1135–1144). ACM.
- Rivera-López, R., Canul-Reich, J., Gámez, J. A., & Puerta, J. M. (2017). OC1-DE: a differential evolution based approach for inducing oblique decision trees. In *Proceedings of the 16th int. conf. on artificial intelligence and soft computing, LNCS 10245* (pp. 427–438). Springer.
- Rodrigo, E. G., Alfaro, J. C., Aledo, J. A., & Gámez, J. A. (2024). Label ranking oblique trees. *Knowledge Based Systems*, 296, Article 111882.
- Rodríguez, J. J., Kuncheva, L. I., & Alonso, C. J. (2006). Rotation forest: A new classifier ensemble method. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(10), 1619–1630, Cited by: 1382.
- Rokach, L. (2010). *Pattern classification using ensemble methods*. Inc.: World Scientific Publishing Co..
- Sagi, O., & Rokach, L. (2018). Ensemble learning: A survey. *WIREs Data Mining Knowledge Discovery*, 8(4).
- Schapire, R. E. (2003). *The boosting approach to machine learning: an overview* (pp. 149–171). New York, NY: Springer New York.
- Shao, Y.-H., Zhang, C.-H., Wang, X.-B., & Deng, N.-Y. (2011). Improvements on twin support vector machines. *IEEE Transactions on Neural Networks*, 22(6), 962–968, Cited by: 417.
- Shotton, J., Girshick, R., Fitzgibbon, A., Sharp, T., Cook, M., Finocchio, M., et al. (2013). Efficient human pose estimation from single depth images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(12), 2821–2840.
- Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90.
- Singh Hada, S., & Carreira-Perpiñán, M. A. (2022). Interpretable image classification using sparse oblique decision trees. In *ICASSP 2022-2022 IEEE international conference on acoustics, speech and signal processing* (pp. 2759–2763).
- Tan, P. J., & Dowe, D. L. (2006). Decision forests with oblique decision trees. In *MICAI 2006: advances in artificial intelligence* (pp. 593–603). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Ting, K. M., & Witten, I. H. (1999). Issues in stacked generalization. *Journal of Artificial Intelligence Research*, 10(1), 271–289.
- Vapnik, V. N. (1995). *The nature of statistical learning theory*. Berlin, Heidelberg: Springer-Verlag.
- Witten, I. H., & Frank, E. (2005). *Data mining: practical machine learning tools and techniques* (2nd ed.). San Francisco: Morgan Kaufmann.
- Wu, X., Kumar, V., Ross Quinlan, J., Ghosh, J., Yang, Q., Motoda, H., et al. (2007). Top 10 algorithms in data mining. *Knowledge and Information Systems*, 14(1), 1–37.
- Yang, B.-B., Shen, S.-Q., & Gao, W. (2019). Weighted oblique decision trees. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33, 5621–5627.
- Yildiz, C., & Alpaydin, E. (2001). Omnivariate decision trees. *IEEE Transactions on Neural Networks*, 12(6), 1539–1546.
- Zaki, M., & Meira, W. (2020). *Data mining and machine learning: fundamental concepts and algorithms* (2nd ed.). New York, USA: Cambridge University Press.
- Zhan, H., Liu, Y., & Xia, Y. (2024). Consistency of oblique decision tree and its boosting and random forest.
- Zhang, G. P. (2000). Neural networks for classification: A survey. *Transactions on Systems Man and Cybernetics Part C*, 30(4), 451–462.
- Zhang, S., Li, X., Zong, M., Zhu, X., & Wang, R. (2018). Efficient knn classification with different numbers of nearest neighbors. *IEEE Transactions on Neural Networks and Learning Systems*, 29(5), 1774–1785.
- Zhang, L., & Suganthan, P. N. (2015). Oblique decision tree ensemble via multisurface proximal support vector machine. *IEEE Transactions Cybernetics*, 45(10), 2165–2176.
- Zhang, L., Zhou, W., Su, T., & Jiao, L. (2007). Decision tree support vector machine. *International Journal on Artificial Intelligence Tools*, 16(1), 1–16.
- Zhou, Z.-H. (2012). *Ensemble methods: foundations and algorithms* (1st ed.). Chapman & Hall/CRC.
- Zhu, B., Farivar, M., & Shoaran, M. (2020). Resot: Resource-efficient oblique trees for neural signal classification.